과제 1번 · SKALA AI 서비스 MINI-PROJECT

Base AI Agent의 RunBook 선택 개선

RAG 기반 RunBook 선택 · 라우팅 기준 확대 · 데이터 확장 · 평가 하네스

2분 — 기능과 결과
무엇을 고쳤나 · Before/After 수치

3분 — 코드 설명
Agent Graph 구성 · MCP · RAG

79번 박건우 (G079 · 광주 3반) · 2026-10-21

github.com/pigpgw/skala-ai-service-mini

장애 유형 판정이 "키워드 첫 매칭"이었다

규칙 순서가 증상 우선순위와 무관했고, 질의문은 아예 보지 않았다.
고치기 전에 기준선부터 재니 12문항 중 4개가 어긋났다.

8/12
keyword 기준선 정확도 = 0.6667

4개
오분류 — 모두 코드 위치로 추적

0
질의문을 읽는 코드 (0줄)

무엇이 틀렸나

- B07 gateway(타임아웃) + payments(CPU)를 함께 보면 알림이 한 덩어리라 cpu가 먼저 잡힘
  `agent/graph.py:195–216`
- B09 memory가 disk보다 앞에 있어 디스크 문제를 메모리 절차로 답함
- B10·B11 판정 함수가 질의문을 읽지 않음 → "디스크 부족"이 지연으로 분류됨
- 매칭 실패 → 무조건 db-down ("아무 장애나 db-down 대응")

```
# agent/graph.py
for keyword, incident_type in _RULES:
    if keyword in text:
        return incident_type   # 첫 매칭
return "db-down"             # 기본값
```

판정 로직 (개선 전)

네 축을 손봤다 — 판정 하나를 바꾸고, 측정할 수 있게 만들었다

1. RAG 기반 RunBook 선택

키워드 첫 매칭 → 런북 10종 임베딩 의미 검색. 확신이 없으면 keyword로 폴백해 기존 동작이 깨지지 않게 했다.

agent/rag/{embedder,index,classifier}.py

2. 라우팅 기준 확대

결정론 백엔드의 정규식이 키워드 7개뿐이라 매뉴얼 예시 질의 7개가 chat으로 떨어졌다. 운영 어휘로 넓혔다.

agent/fake_llm.py::route

3. 데이터 확장

모니터링 6 → 12시스템, 런북 6 → 10종(신규 유형 4종). 파일 선택은 한 곳에서만.

agent/data_profile.py · data/monitoring_extended.json

4. 평가 하네스

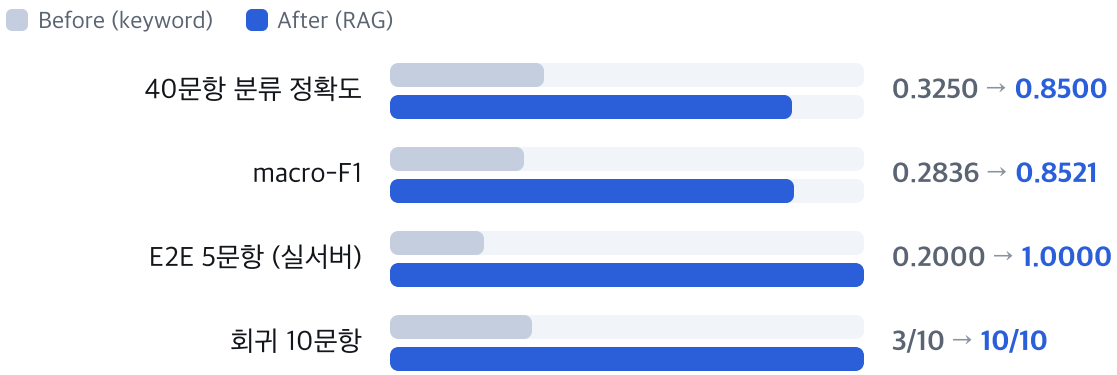
실행 1회 = 폴더 1개(조건·지표·입출력), 40문항 라벨 동결, 회귀 10문항 실행기, 문서 숫자 ↔ 원본 자동 대조.

eval/run_eval.py · tests/ (65개)

개선 후 회귀 질의 10개: 3/10 → 10/10

매뉴얼 9쪽이 알려준 테스트 질의로 자기 자신을 테스트할 수 없던 상태였다.

같은 입력, 같은 조건 — 2회 실행한 결과



비용과 부작용

지표	Before	After	차이
분류 지연 p50	0.0ms	32.43ms	+32.43ms
비용 (USD)	0.000000	0.000000	0
retry 횟수	1	1	0
keyword 폴백률	100.00%	15.00%	**−85.00%p**

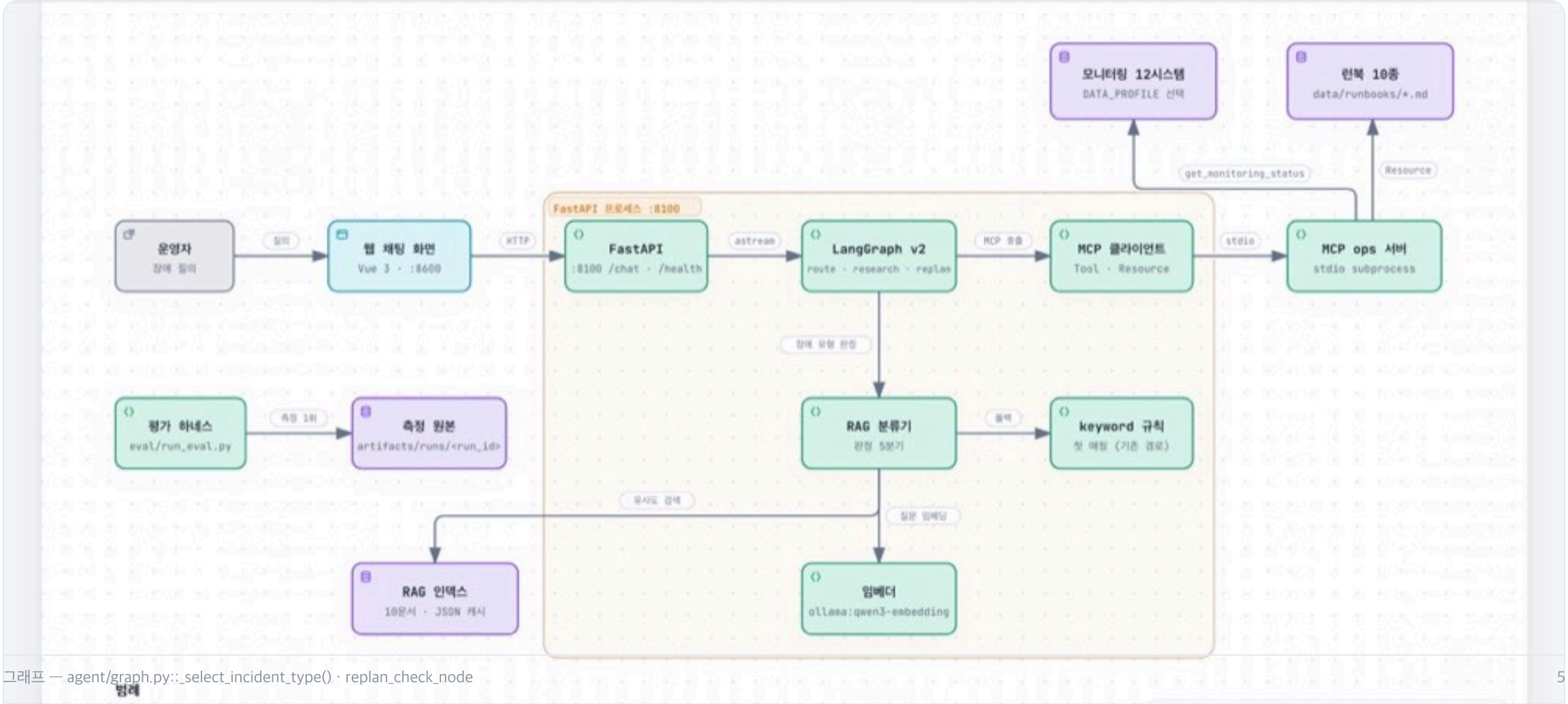
한계도 그대로

데이터 확장(6→12)은 분류 정확도를 0 올렸다(base 34/40 ≡ extended 34/40). 바뀐 건 조사 커버리지(unknown 18.75% → 0%). margin 게이트는 2문항을 아꼈다.

근거 화면·원본: doc/reports/evidence/before_after.md

그래프는 그대로 — 판정 한 줄만 갈아끼웠다

route → plan → gate → research → replan_check → analysis → write → validate → finalize.
replan_check가 런북을 추가로 조회하는 지점이고, 그 판단을 RAG로 바꿨다.



그래프 — agent/graph.py::_select_incident_type() · replan_check_node

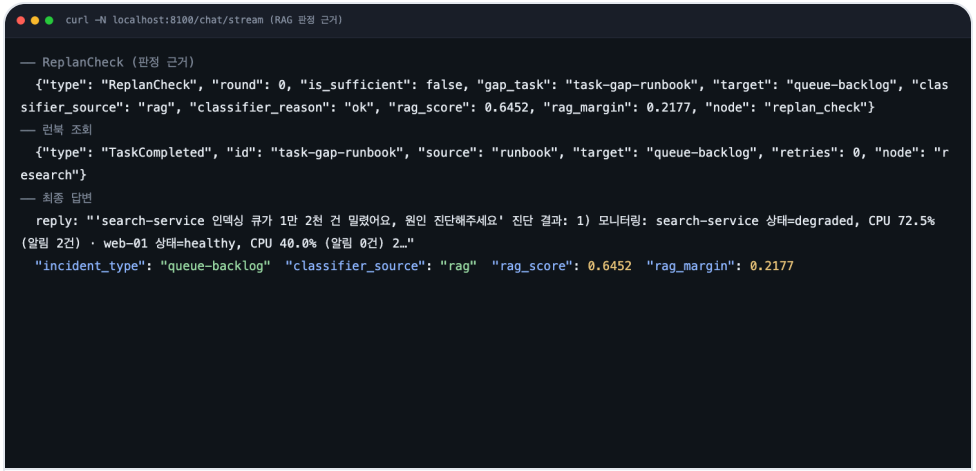
임베딩 검색 + 확신 없으면 기존 경로로

```
# gate: 확신 없으면 keyword로
if not enabled:              → keyword
if index is None:              → keyword
vector = embedder.embed([text])[0]
top1, top2 = index.search(vector)
if top1 < min_score:       → keyword  (0.50)
if top1-top2 < min_margin:  → keyword  (0.03)
return top1                    # rag
```

agent/rag/classifier.py — 판정 5분기

입력 4블록

[사용자 질문][시스템][상태][알림] — 질의문을 넣는 이유는 기존 오분류 2개가 "질의를 안 본 것"이었기 때문.



실제 SSE 출력 — classifier_source=rag · rag_score=0.6452 · rag_margin=0.2177

게이트가 안전장치다

RAG가 확신하지 못하면 예전 경로로 내려간다. 그래서 기존 동작이 깨지지 않는다는 걸 숫자로 확인할 수 있었다.

조회는 MCP로, 데이터 선택은 한 곳에서

```
@mcp.tool      get_monitoring_status(system_name)
@mcp.resource  get_incident_runbook(incident_type)
@mcp.prompt    incident_analysis_prompt(system, symptom)

# agent/data_profile.py — 파일 선택의 단일 지점
DATA_PROFILE = base | extended    # 6시스템 | 12시스템
```

mcp_server.py — ops 서버

경로가 다섯 곳에 흩어져 있으면

시스템만 늘고 실제로 조사하는 대상은 여섯 개로 잘린다 — 이름만 늘고 조사는 그대로인 "가짜 확장"이 된다.

- Tool / Resource / Prompt 를 구분해 쓴다
  모니터링 조회 · 런북 본문 · 분석 절차
- stdio subprocess 로 붙는다
  agent/mcp_client.py — list → find → read 순서로 탐색
- decoy 서버는 쓰지 않는다
  mcp_server_docs.py — 쓰지 않음이 설계 의도
- RAG는 의존성 주입
  SupervisorDeps.rag — 노드가 환경변수를 직접 읽지 않는다
- LangSmith 트레이싱 경로 확보
  노드에 @traceable — 실 API 키 없는 환경이라 캡처는 못 했다

마무리

판정을 바꾼 게 아니라,
판단의 근거를 남기고 측정할 수 있게 바꾼 것

0.3250 → 0.8500
40문항 분류 정확도

0.2836 → 0.8521
macro-F1

3/10 → 10/10
회귀 질의 (매뉴얼 9쪽)

배운 점

- RAG의 핵심은 임베딩보다 입력 정의와 게이트였다
- "개선"이라고 말하려면 기준선을 먼저 재야 한다
- 근거를 화면에 남기면 설명하지 않아도 확인된다

다음에 하고 싶은 것

- 실제 장애 티켓으로 라벨 세트 재구축 (지금은 자기 평가)
- LLM-as-judge 평가 추가 · Research 서브그래프 분리
- 실 OpenAI 백엔드로 비용·지연 재측정